FoodSense — Publishing Guide

- FoodSense — App Store & Play Store Publishing Guide
  - Table of Contents
  - 1. Prerequisites
    - Accounts You Need (Paid)
    - Tools to Install
  - 2. Backend Deployment
    - Step 2.1 — Set up DNS
    - Step 2.2 — Provision DigitalOcean Droplet
    - Step 2.3 — Clone and Configure
    - Step 2.4 — Launch Stack
    - Step 2.5 — Nginx Reverse Proxy + HTTPS
    - Step 2.6 — Lockdown
    - Step 2.7 — Host Privacy Policy & Terms
    - Step 2.8 — Database Backups
  - 3. iOS — App Store Publishing
    - Step 3.1 — Apple Developer Account
    - Step 3.2 — Create App ID
    - Step 3.3 — Configure Xcode Project
    - Step 3.4 — Increment Version & Build
    - Step 3.5 — Info.plist Keys (auto-generated — verify in Build Settings)
    - Step 3.6 — App Transport Security
    - Step 3.7 — Create App in App Store Connect
    - Step 3.8 — App Privacy Nutrition Labels
    - Step 3.9 — Privacy Manifest File (Xcode 15+)
    - Step 3.10 — Age Rating
    - Step 3.11 — Pricing & Availability
    - Step 3.12 — Archive & Upload
    - Step 3.13 — TestFlight (Internal Beta)
    - Step 3.14 — Submit for Review
  - 4. Android — Play Store Publishing
    - Step 4.1 — Play Console Account
    - Step 4.2 — Generate Release Signing Keystore
    - Step 4.3 — Configure Signing in build.gradle.kts
    - Step 4.4 — Bundle Identifier
    - Step 4.5 — Configure AndroidManifest.xml
    - Step 4.6 — Build Release AAB (Android App Bundle)
    - Step 4.7 — Create App in Play Console
    - Step 4.8 — Data Safety Declaration (Required)
    - Step 4.9 — Content Rating
    - Step 4.10 — Target Audience
    - Step 4.11 — Store Listing
    - Step 4.12 — App Bundle Explorer & Release
    - Step 4.13 — Internal Testing Track (Recommended First)
    - Step 4.14 — Play Store Review
  - 5. Store Listing Assets
    - Icons
    - Screenshots (Both Stores)
    - Feature Graphic (Play Store only)
    - App Store Description (iOS)
    - Play Store Full Description (4000 chars)

- 6. Legal & Privacy
  - Privacy Policy (Host at foodsensescale.tech/privacy)
  - Terms of Service (Host at foodsensescale.tech/terms)
- 7. Post-Launch
  - Monitoring
  - Respond to Reviews
  - Version Updates
  - Staged Rollout (Android)
- Final Checklist Before Submission
  - Both Platforms
  - iOS-Specific
  - Android-Specific

FoodSense — App Store & Play Store Publishing Guide

Complete step-by-step instructions to publish FoodSense on both stores. Backend domain: **foodsensescale.tech/api**

Table of Contents

1. Prerequisites
2. Backend Deployment
3. iOS — App Store Publishing
4. Android — Play Store Publishing
5. Store Listing Assets
6. Legal & Privacy
7. Post-Launch

---

1. Prerequisites

Accounts You Need (Paid)

Account	Cost	Purpose
Apple Developer Program	$99/year	App Store publishing
Google Play Console	$25 one-time	Play Store publishing
Domain `foodsensescale.tech`	~$15/year	Backend + privacy policy hosting
DigitalOcean Droplet	$6-12/month	Server hosting
Firebase (Spark plan)	Free	Auth + Analytics + Crashlytics

Tools to Install

- **Xcode 26.4+** (for iOS builds, signing, App Store upload)
- **Android Studio Koala+** (for Android builds, signing, Play upload)
- **Transporter.app** (Mac App Store — for IPA uploads)
- **gcloud CLI** (optional, for Firebase management)

- **Docker Desktop** (for server deployment)

---

2. Backend Deployment

All API calls from both apps hit `https://foodsensescale.tech/api/*`.

Step 2.1 — Set up DNS

At your domain registrar for `foodsensescale.tech`:

```
Type    Name    Value                    TTL
A       @       <your-droplet-ip>        300
A       api     <your-droplet-ip>        300
CNAME   www     foodsensescale.tech      300
```

Verify: `dig +short foodsensescale.tech` should return your droplet IP.

Step 2.2 — Provision DigitalOcean Droplet

Minimum specs: **2 GB RAM, 2 vCPU, 50 GB SSD** ($12/month)

```
# SSH into the fresh droplet
ssh root@<droplet-ip>

# Install Docker + Docker Compose
curl -fsSL https://get.docker.com | sh
apt install -y docker-compose-plugin git nginx certbot python3-certbot-nginx
systemctl enable docker
```

Step 2.3 — Clone and Configure

```
cd /root
git clone https://github.com/nitinyadav26/FoodDetectionApp.git
cd FoodDetectionApp/server

# Copy your secrets (upload these separately, NEVER commit them)
# - firebase-service-account.json  (from Firebase Console)
# - .env  (create from .env.example)
```

Create `/root/FoodDetectionApp/server/.env`:

```
DATABASE_URL="postgresql://postgres:CHANGE_ME_RANDOM_PASSWORD@postgres:5432/foodsense?
schema=public"
POSTGRES_PASSWORD="CHANGE_ME_RANDOM_PASSWORD"
REDIS_URL="redis://:CHANGE_ME_REDIS_PASSWORD@redis:6379"
REDIS_PASSWORD="CHANGE_ME_REDIS_PASSWORD"

# Generate: openssl rand -hex 32
JWT_SECRET="CHANGE_ME_64_CHAR_HEX_STRING"

# Optional: server-side Gemini proxy (only if you want to offer cloud AI without users
bringing their own key)
GEMINI_API_KEY=""

# Firebase
FIREBASE_SERVICE_ACCOUNT_PATH="./firebase-service-account.json"
FIREBASE_STORAGE_BUCKET="your-project-id.firebasestorage.app"
```

```
# Environment
PORT=3000
NODE_ENV=production
```

Generate secrets:

```
# Postgres + Redis passwords
openssl rand -base64 24
openssl rand -base64 24

# JWT secret
openssl rand -hex 32
```

Step 2.4 — Launch Stack

```
cd /root/FoodDetectionApp/server
docker compose up -d --build

# Run database migrations
docker compose exec app npx prisma migrate deploy
docker compose exec app npx prisma db seed  # if you have seed data

# Verify
docker compose logs -f app
curl http://localhost:3000/health
# Should return: {"status":"ok","timestamp":"..."}
```

Step 2.5 — Nginx Reverse Proxy + HTTPS

Create `/etc/nginx/sites-available/foodsensescale.tech`:

```
server {
    listen 80;
    server_name foodsensescale.tech www.foodsensescale.tech;

    # Let's Encrypt verification
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

server {
    listen 443 ssl http2;
    server_name foodsensescale.tech www.foodsensescale.tech;

    ssl_certificate /etc/letsencrypt/live/foodsensescale.tech/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/foodsensescale.tech/privkey.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    client_max_body_size 50M;  # Allow food image uploads

    # Serve privacy policy + terms as static HTML
    location = /privacy {
```

```
        alias /var/www/foodsense/privacy.html;
    }
    location = /terms {
        alias /var/www/foodsense/terms.html;
    }

    # API routes proxy to Node backend
    location /api/ {
        proxy_pass http://127.0.0.1:3000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
        proxy_connect_timeout 10s;
    }

    # Auth verify endpoint (not under /api prefix)
    location /auth/ {
        proxy_pass http://127.0.0.1:3000/auth/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # Health check
    location = /health {
        proxy_pass http://127.0.0.1:3000/health;
    }

    # Social aliases (legacy)
    location /social/ {
        proxy_pass http://127.0.0.1:3000/social/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
    }
}
```

Enable and obtain certificate:

```
ln -s /etc/nginx/sites-available/foodsensescale.tech /etc/nginx/sites-enabled/
nginx -t  # test config
systemctl reload nginx

# Get SSL certificate
certbot --nginx -d foodsensescale.tech -d www.foodsensescale.tech --non-interactive --agree-
        tos -m your-email@example.com

# Auto-renewal is installed via systemd timer
systemctl list-timers certbot*
```

Step 2.6 — Lockdown

```
# Only expose ports 80, 443, 22 (SSH)
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
```

```
# Remove public port 3000 binding from docker-compose.yml
# Change "3000:3000" → "127.0.0.1:3000:3000" so only nginx can reach it
```

Edit `server/docker-compose.yml`:

```yaml
app:
  build: .
  ports:
    - "127.0.0.1:3000:3000"  # Changed from "3000:3000"
  ...
```

Then: `docker compose up -d`

Step 2.7 — Host Privacy Policy & Terms

```bash
mkdir -p /var/www/foodsense
cp /root/FoodDetectionApp/docs/privacy.html /var/www/foodsense/
cp /root/FoodDetectionApp/docs/terms.html /var/www/foodsense/
chown -R www-data:www-data /var/www/foodsense
```

Verify: - https://foodsensescale.tech/privacy → Privacy Policy - https://foodsensescale.tech/terms → Terms of Service - https://foodsensescale.tech/health → `{"status":"ok"}` - https://foodsensescale.tech/api/friends → 401 (auth required — correct)

Step 2.8 — Database Backups

```bash
# Daily cron backup to S3/Backblaze/local
crontab -e

# Add:
0 3 * * * docker compose -f /root/FoodDetectionApp/server/docker-compose.yml exec -T
        postgres pg_dump -U postgres foodsense | gzip > /root/backups/foodsense-$(date
        +\%Y\%m\%d).sql.gz
0 4 * * * find /root/backups -name "*.sql.gz" -mtime +30 -delete
```

---

3. iOS — App Store Publishing

Step 3.1 — Apple Developer Account

1. Enroll at https://developer.apple.com/programs/ ($99/year)
2. Accept agreements in App Store Connect
3. Add yourself to certificates, identifiers, and provisioning in Xcode:
    - Xcode → Settings → Accounts → + → sign in with Apple ID
    - Select your team

Step 3.2 — Create App ID

At https://developer.apple.com/account/resources/identifiers:

1. **Identifiers** → + → **App IDs** → **App**
2. Description: `FoodSense`
3. Bundle ID: Explicit → `com.foodsense.ios` (must match `PRODUCT_BUNDLE_IDENTIFIER` in Xcode)
4. Capabilities (check all needed):

    - ☑ HealthKit
    - ☑ Push Notifications
    - ☑ Associated Domains (for universal links, optional)
5. Continue → Register

Step 3.3 — Configure Xcode Project

Open `FoodDetectionApp.xcworkspace`:

1. Select target **FoodDetectionApp → Signing & Capabilities**
2. Team: Your Apple Developer team
3. Bundle Identifier: `com.foodsense.ios`
4. Signing Certificate: Apple Development (debug) / Apple Distribution (release)
5. Verify these capabilities are added:
    - HealthKit
    - Push Notifications
    - Background Modes → Background fetch, Remote notifications
    - Associated Domains (if using universal links)

Step 3.4 — Increment Version & Build

In project settings → **General** tab: - **Version**: `1.0.0` (your marketing version) - **Build**: `1` (increment this for every TestFlight upload)

Step 3.5 — Info.plist Keys (auto-generated — verify in Build Settings)

Required usage descriptions:

```
INFOPLIST_KEY_NSCameraUsageDescription = "FoodSense uses your camera to scan food and analyze nutrition."
INFOPLIST_KEY_NSPhotoLibraryUsageDescription = "FoodSense needs photo access to analyze food images from your library."
INFOPLIST_KEY_NSHealthShareUsageDescription = "FoodSense reads your health data to personalize nutrition tracking."
INFOPLIST_KEY_NSHealthUpdateUsageDescription = "FoodSense saves your food logs to Apple Health."
INFOPLIST_KEY_NSBluetoothAlwaysUsageDescription = "FoodSense connects to smart scales for weight tracking."
INFOPLIST_KEY_NSMicrophoneUsageDescription = "FoodSense uses the microphone for voice food logging."
INFOPLIST_KEY_NSSpeechRecognitionUsageDescription = "FoodSense transcribes your voice to log food hands-free."
INFOPLIST_KEY_NSUserTrackingUsageDescription = "This helps us improve FoodSense. We don't share your data with third parties."
```

Step 3.6 — App Transport Security

Your app uses `https://foodsensescale.tech` — no ATS exception needed. If you still have the cleartext exception from dev, remove it:

In Info build settings, do NOT include `NSAppTransportSecurity → NSAllowsArbitraryLoads = YES`.

Step 3.7 — Create App in App Store Connect

https://appstoreconnect.apple.com → **My Apps** → + → **New App**

- Platform: iOS
- Name: **FoodSense — AI Nutrition Tracker** (30 char max)
- Primary Language: English (U.S.)
- Bundle ID: `com.foodsense.ios` (from step 3.2)
- SKU: `FOODSENSE001`
- User Access: Full Access

Step 3.8 — App Privacy Nutrition Labels

Required by Apple since 2020. In App Store Connect → your app → **App Privacy**:

Declare data collected:

Data Type	Linked to User	Used for Tracking	Purpose
Email	Yes	No	App functionality (auth)
Name	Yes	No	App functionality
Photos	No (on-device)	No	App functionality (food detection)
Health & Fitness	Yes	No	App functionality
Usage Data	Yes	No	Analytics
Diagnostics	Yes	No	Crash reports

Critical: State clearly that food photos are processed **on-device** (when using Gemma) and **never uploaded** unless user provides their own Gemini API key.

Step 3.9 — Privacy Manifest File (Xcode 15+)

Create `FoodDetectionApp/PrivacyInfo.xcprivacy`:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
        "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>NSPrivacyTracking</key>
    <false/>
    <key>NSPrivacyTrackingDomains</key>
    <array/>
    <key>NSPrivacyCollectedDataTypes</key>
    <array>
        <dict>
            <key>NSPrivacyCollectedDataType</key>
            <string>NSPrivacyCollectedDataTypeEmailAddress</string>
            <key>NSPrivacyCollectedDataTypeLinked</key>
            <true/>
            <key>NSPrivacyCollectedDataTypeTracking</key>
            <false/>
            <key>NSPrivacyCollectedDataTypePurposes</key>
            <array>
                <string>NSPrivacyCollectedDataTypePurposeAppFunctionality</string>
            </array>
        </dict>
        <dict>
            <key>NSPrivacyCollectedDataType</key>
            <string>NSPrivacyCollectedDataTypePhotos</string>
            <key>NSPrivacyCollectedDataTypeLinked</key>
```

```xml
            <false/>
            <key>NSPrivacyCollectedDataTypeTracking</key>
            <false/>
            <key>NSPrivacyCollectedDataTypePurposes</key>
            <array>
                <string>NSPrivacyCollectedDataTypePurposeAppFunctionality</string>
            </array>
        </dict>
        <dict>
            <key>NSPrivacyCollectedDataType</key>
            <string>NSPrivacyCollectedDataTypeHealthAndFitness</string>
            <key>NSPrivacyCollectedDataTypeLinked</key>
            <true/>
            <key>NSPrivacyCollectedDataTypeTracking</key>
            <false/>
            <key>NSPrivacyCollectedDataTypePurposes</key>
            <array>
                <string>NSPrivacyCollectedDataTypePurposeAppFunctionality</string>
            </array>
        </dict>
    </array>
    <key>NSPrivacyAccessedAPITypes</key>
    <array>
        <dict>
            <key>NSPrivacyAccessedAPIType</key>
            <string>NSPrivacyAccessedAPICategoryFileTimestamp</string>
            <key>NSPrivacyAccessedAPITypeReasons</key>
            <array>
                <string>C617.1</string>
            </array>
        </dict>
        <dict>
            <key>NSPrivacyAccessedAPIType</key>
            <string>NSPrivacyAccessedAPICategoryUserDefaults</string>
            <key>NSPrivacyAccessedAPITypeReasons</key>
            <array>
                <string>CA92.1</string>
            </array>
        </dict>
    </array>
</dict>
</plist>
```

Drag this file into the Xcode project (check “Copy items if needed” and add to `FoodDetectionApp` target).

Step 3.10 — Age Rating

In App Store Connect → your app → **App Information** → **Age Rating**:

Answer the questionnaire. For a nutrition app: - Unrestricted Web Access: No - Violence: None - Mature themes: None - Gambling: None

Expected rating: 4+ (all ages).

Step 3.11 — Pricing & Availability

- **Price:** Free
- **Availability:** All territories

- **Pre-Order:** No

Step 3.12 — Archive & Upload

1. Select device: **Any iOS Device (arm64)** (not a simulator)
2. Product → Archive
3. Wait for build to complete (~5 min)
4. Window → Organizer → **Distribute App** → **App Store Connect** → **Upload** → Automatic signing → Upload
5. Wait 15-30 min for Apple processing
6. You'll get an email when the build is ready

Step 3.13 — TestFlight (Internal Beta)

Before public release, test via TestFlight:

1. App Store Connect → your app → **TestFlight** tab
2. Add yourself + team members as **Internal Testers**
3. Select your build → **Provide Export Compliance** (No, not using encryption beyond standard)
4. Install TestFlight app on your iPhone → accept invite → test

Step 3.14 — Submit for Review

1. **App Information** → fill all required fields
2. **Pricing and Availability** → set
3. Go to version **1.0.0** page:
    - **Screenshots** (see Section 5)
    - **Promotional text** (170 chars, updatable without re-submit)
    - **Description** (see Section 5)
    - **Keywords** (100 chars, comma-separated)
    - **Support URL:** `https://foodsensescale.tech`
    - **Marketing URL** (optional): `https://foodsensescale.tech`
    - **Privacy Policy URL:** `https://foodsensescale.tech/privacy`
4. **Build** → select the TestFlight build
5. **App Review Information:**
    - Contact info + demo account credentials (create a test Firebase account)
    - Notes: "This app runs Gemma 4 AI on-device via llama.cpp. No data leaves the device unless the user provides their own Gemini API key. Food photos are NEVER uploaded."
6. **Version Release:** Manual release (so you control the launch moment)
7. Click **Save** → **Submit for Review**

Review takes 24-48 hours typically. Be prepared for potential rejections about: - Missing demo credentials - Unclear "free" pricing (they think "free" with cloud AI is misleading — emphasize on-device) - Vague descriptions of AI features

---

4. Android — Play Store Publishing

Step 4.1 — Play Console Account

1. Pay $25 one-time at https://play.google.com/console/signup
2. Verify identity (24-48 hours)

3. Accept developer distribution agreement

Step 4.2 — Generate Release Signing Keystore

On your dev machine (one-time, NEVER lose this file):

```
cd ~/Documents  # or wherever you want to store it
keytool -genkey -v -keystore foodsense-release.keystore \
  -alias foodsense -keyalg RSA -keysize 2048 -validity 10000

# Answer prompts:
# Password: choose a strong password
# First/last name: Nitin Yadav
# Org: FoodSense
# City, State, Country: your info
```

Back up `foodsense-release.keystore` in 2 places. If you lose it, you can never update the app.

Step 4.3 — Configure Signing in build.gradle.kts

Add to `android/gradle.properties` (NOT committed — already gitignored):

```
RELEASE_STORE_FILE=/Users/YOU/Documents/foodsense-release.keystore
RELEASE_STORE_PASSWORD=your-keystore-password
RELEASE_KEY_ALIAS=foodsense
RELEASE_KEY_PASSWORD=your-key-password
```

The `android/app/build.gradle.kts` already reads these props (lines 44-52).

Step 4.4 — Bundle Identifier

Verify `android/app/build.gradle.kts`:

```
applicationId = "com.foodsense.android"  // Must be unique on Play Store
versionCode = 1     // Integer, increment every release
versionName = "1.0.0"  // User-facing
```

Step 4.5 — Configure AndroidManifest.xml

Ensure these permissions are declared (they should already be):

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

<!-- Health Connect -->
<uses-permission android:name="android.permission.health.READ_STEPS" />
<uses-permission android:name="android.permission.health.READ_HYDRATION" />
<uses-permission android:name="android.permission.health.WRITE_STEPS" />
```

Step 4.6 — Build Release AAB (Android App Bundle)

Play Store requires `.aab` format (not `.apk`).

```
cd /Users/nitin/Desktop/FoodDetectionApp/android
./gradlew bundleRelease

# Output: android/app/build/outputs/bundle/release/app-release.aab
```

Verify the AAB is signed:

```
jarsigner -verify -verbose -certs app/build/outputs/bundle/release/app-release.aab | head
# Should say: "jar verified."
```

Step 4.7 — Create App in Play Console

https://play.google.com/console → **Create app**

- App name: **FoodSense — AI Nutrition Tracker**
- Default language: English (US)
- App or game: App
- Free or paid: Free
- Declarations: Check all relevant boxes (privacy policy, ads, etc.)

Step 4.8 — Data Safety Declaration (Required)

Play Console → your app → **Policy** → **Data safety**:

Answer truthfully:

Data collected and shared: | Category | Collected | Shared | Processed in Transit | Optional | Purpose | |— — —-| — — —–|— —–|— — — — — — —|— — —-|— — —| | Email | Yes | No | Yes (TLS) | No | Account mgmt | | Name | Yes | No | Yes | Yes | Account mgmt | | Photos | No* | No | No (on-device) | N/A | App functionality | | Health info | Yes | No | Yes | Yes | App functionality | | App performance | Yes | No | Yes | Yes | Analytics | | Crash logs | Yes | No | Yes | No | Diagnostics |

*Photos are processed on-device via Gemma 4 / YOLO. Only uploaded if user provides a Gemini API key, and even then only to Google's Gemini API, not our servers.

Data encryption: Yes, in transit (HTTPS) and at rest (platform keychain + EncryptedSharedPreferences for API keys).

Data deletion: Users can delete account via Settings → Delete Account. Server-side cascading delete removes all associated data.

Step 4.9 — Content Rating

Complete the IARC questionnaire. For a nutrition tracker: - Violence: None - Gambling: None - Profanity: None - Controlled substances: None - User-generated content: Yes (challenges, feed) - **Expected rating: Everyone (E)**

Step 4.10 — Target Audience

- Target age: 13+
- Appeals to children: No

Step 4.11 — Store Listing

Play Console → your app → **Grow** → **Store presence** → **Main store listing**:

- **App name**: FoodSense — AI Nutrition Tracker
- **Short description** (80 chars): On-device AI calorie tracker. No cloud, no subscription, 100% private.
- **Full description**: (see Section 5)
- **App icon**: 512x512 PNG (see Section 5)
- **Feature graphic**: 1024x500 PNG
- **Screenshots**: 2-8 per device type (phone, 7" tablet, 10" tablet)
- **Contact details**:
  - Email: your-support@foodsensescale.tech
  - Website: https://foodsensescale.tech
  - Privacy Policy: https://foodsensescale.tech/privacy

Step 4.12 — App Bundle Explorer & Release

1. Play Console → **Production** → **Create new release**
2. **Upload** your `app-release.aab`
3. Play Console handles signing with Play App Signing
4. **Release name:** `1.0.0 (1)`
5. **Release notes**: "Initial release of FoodSense — the first open-source nutrition tracker with on-device AI."
6. **Review release** → **Start rollout to Production**

Step 4.13 — Internal Testing Track (Recommended First)

Before production: 1. Play Console → **Testing** → **Internal testing** 2. Create a release → upload AAB → Add testers (Google accounts) 3. Share opt-in URL with testers → test for a few days 4. When satisfied, promote to production

Step 4.14 — Play Store Review

Review takes 1-7 days typically. Common rejection reasons: - Missing/broken privacy policy URL - Missing data safety declarations - Permissions not justified (explain each in description) - App crashes on launch

---

5. Store Listing Assets

Icons

iOS App Icon: 1024x1024 PNG (no transparency, no rounded corners — Apple applies them) - Place in `FoodDetectionApp/Assets.xcassets/AppIcon.appiconset/`

Android App Icon: - 512x512 PNG for Play Store listing - Adaptive icon XML in `android/app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml` (already exists)

Screenshots (Both Stores)

iPhone (required — 6.7" display): 1290x2796 or 1320x2868 px

Capture 5-6 screenshots showing: 1. **Hero:** Scan camera pointed at food with "Powered by On-Device AI" badge visible 2. **Results:** Food detected with calories/macros 3. **AI Coach:** Chat conversation with Gemma 4. **Meal Plan:** 7-day meal plan screen 5. **Settings:** AI Configuration showing "Active Provider: On-Device AI" 6. **Dashboard:** Daily nutrition dashboard

Android Phone: 1080x1920 or similar (16:9 or 9:16 ratio, min 320px short side, max 3840px)

Feature Graphic (Play Store only)

1024x500 PNG. Should visually communicate “on-device AI” — include: - App icon - “Runs Gemma 4 On-Device” tagline - Sample food photo with detected nutrition overlay

App Store Description (iOS)

```
FoodSense is the first open-source calorie tracker that runs Google's Gemma 4 AI model
entirely on your phone. No cloud. No API key required. No subscription.

Your food photos NEVER leave your device.

★ WHAT MAKES FOODSENSE DIFFERENT ★

• On-device AI: Gemma 4 E2B runs locally via llama.cpp
• Privacy-first: Food photos processed on-device, never uploaded
• Free forever: No subscription, no hidden fees
• Offline: Works in airplane mode after model download
• Indian food: 94+ Indian dishes detected (Biryani, Dal, Samosa, etc.)
• Open source: Fully auditable code on GitHub

★ FEATURES ★

• Scan food with camera for instant calorie/macro analysis
• AI Health Coach powered by Gemma 4
• 7-day meal plan generation
• Weekly nutrition insights with charts
• Apple Health integration
• BLE smart scale support
• Friends, challenges, leaderboards

★ HOW IT WORKS ★

1. Point camera at food
2. YOLO detector identifies the dish (instant)
3. Gemma 4 estimates calories, protein, carbs, fats
4. Log it to your daily intake

★ PRIVACY ★

FoodSense is built privacy-first. The Gemma 4 AI model runs entirely on your iPhone using
llama.cpp with Metal GPU acceleration. Your food photos, nutrition data, and personal stats
never leave your device unless you explicitly choose to sync for multi-device support.

Optional: Enter your own Google Gemini API key in Settings for cloud-based analysis with
higher accuracy. Even then, your data goes directly to Google's API — we never see it.

★ REQUIREMENTS ★

• iOS 17 or later
• iPhone with A14 Bionic or newer (iPhone 12+)
• 5 GB free storage (3.2 GB AI model + app data)

★ OPEN SOURCE ★

Source code: https://github.com/nitinyadav26/FoodDetectionApp
License: Apache 2.0

Join the community pushing on-device AI forward.
```

Keywords (100 chars):

```
calorie,nutrition,food scanner,indian food,gemma,AI,on-device,offline,privacy,macros,diet
```

Play Store Full Description (4000 chars)

Use the same text as App Store, extend with:

```
[...same core content...]

★ WHY ON-DEVICE AI? ★

Most calorie trackers upload every photo you take to their cloud. We think that's wrong.
FoodSense proves that modern phones are powerful enough to run state-of-the-art AI models
directly — no cloud needed.

When you download FoodSense, you get a 3.2 GB AI model (Gemma 4 E2B by Google DeepMind) that
runs entirely on your phone's GPU. No internet required for AI features.

★ BUILT FOR INDIAN CUISINE ★

Most calorie trackers were built for Western food. FoodSense includes a custom YOLO model
trained on 94+ Indian dishes including:
Biryani, Dal, Sambhar, Idli, Dosa, Paneer, Chole, Rajma, Poha, Khichdi, Samosa, Gulab Jamun,
Jalebi, Rasgulla, Vada, Chaat, and many more.

★ FREE, FOREVER ★

No subscription. No freemium limits. No ads. If you want to support the project, star us on
GitHub.

[...rest of core content...]
```

---

6. Legal & Privacy

Privacy Policy (Host at foodsensescale.tech/privacy)

Your existing `docs/privacy.html` needs these sections:

1. **What data we collect:**
   - Account: email, display name (optional)
   - Health data: via Apple Health / Health Connect (with permission)
   - Food logs: stored locally, optionally synced to your account
   - Analytics: Firebase Analytics (anonymous usage stats, crash reports)
2. **What we DON'T collect:**
   - Food photos (processed on-device, never uploaded)
   - Contents of AI Coach conversations (on-device)
   - Your Gemini API key (stored in iOS Keychain / Android EncryptedSharedPreferences)
3. **Where data goes:**
   - Your own device (primary storage)
   - Our servers at foodsensescale.tech (optional, for multi-device sync)
   - Firebase (auth, analytics, crash reports)

    - Google Gemini API (only if you provide your own key)

4. **Third parties:** Firebase (Google), Apple HealthKit, Health Connect

5. **Your rights:** Export data, delete account, revoke permissions anytime

6. **Children:** Not directed at children under 13 (required for both stores)

7. **Contact:** your-email@foodsensescale.tech

Terms of Service (Host at foodsensescale.tech/terms)

Standard terms covering: - Service description - User conduct - AI-generated content disclaimer ("calorie estimates are approximate") - No warranty (medical disclaimer: "consult a dietitian for real advice") - Indemnification, limitation of liability - Governing law

Both HTML files already exist in `docs/`. Update email addresses and dates.

---

7. Post-Launch

Monitoring

- **Crashlytics:** Firebase Console → Crashlytics → watch for crashes in first 48h
- **Analytics:** Firebase Console → Analytics → track DAU, retention, feature usage
- **Server logs:** `ssh droplet` → `docker compose logs -f app` → watch for errors
- **Uptime:** Set up https://uptimerobot.com (free) to ping `foodsensescale.tech/health` every 5 min

Respond to Reviews

Reply to every review in first 2 weeks. Positive reviews boost ranking; responding to negative ones often flips them.

Version Updates

iOS: Increment build number → Archive → Upload → Submit for Review → Manual release **Android:** Increment versionCode → `./gradlew bundleRelease` → upload AAB → Create release → Rollout

Staged Rollout (Android)

Don't release to 100% immediately. In Play Console → Production → Release → roll out to 20% first, monitor for 24h, then 50%, then 100%.

Final Checklist Before Submission

Both Platforms

- [ ] Privacy policy live at https://foodsensescale.tech/privacy (200+ OK)
- [ ] Terms live at https://foodsensescale.tech/terms (200+ OK)
- [ ] Backend deployed at https://foodsensescale.tech/api (responds 200 on /health, 401 on protected routes)

- [ ] Screenshots captured (5-6 per store)
- [ ] App icon (1024x1024)
- [ ] Feature graphic for Play Store (1024x500)
- [ ] App description written and proofread
- [ ] Keywords chosen (iOS: 100 chars)
- [ ] Firebase project configured for both iOS + Android bundle IDs
- [ ] Firebase Auth method enabled (Anonymous + Email/Password)
- [ ] Crashlytics initialized (both platforms)

iOS-Specific

- [ ] Apple Developer account enrolled ($99 paid)
- [ ] App ID created with HealthKit + Push capabilities
- [ ] Xcode project configured with correct bundle ID + signing
- [ ] PrivacyInfo.xcprivacy file added to target
- [ ] Version 1.0.0, Build 1 (or higher)
- [ ] Archive uploaded to App Store Connect
- [ ] App Privacy Nutrition Labels declared
- [ ] Content rating: 4+
- [ ] Demo account credentials prepared for App Review

Android-Specific

- [ ] Play Console account ($25 paid + identity verified)
- [ ] Release keystore generated + backed up (`foodsense-release.keystore`)
- [ ] `RELEASE_STORE_FILE` etc. set in gradle.properties (NOT committed)
- [ ] `./gradlew bundleRelease` produces signed AAB
- [ ] Data Safety declaration completed
- [ ] Content rating: Everyone
- [ ] Target audience set (13+)
- [ ] All required screenshots uploaded
- [ ] Internal testing track configured

Ship it. The Gemma 4 hype window is closing fast.